THE COMPLETE CONCEPT MAP

AI Engineering

Every major concept an AI engineer needs to understand

From how LLMs work under the hood — through prompting, RAG, fine-tuning, agents, and evaluation — to the 2025–2026 frontier of reasoning models, million-token context, and agentic systems.

What this course is (and who it's for)

AI engineering is the discipline of building real applications on top of foundation models (large language models and their multimodal cousins). It's distinct from classic ML engineering: you usually don't train the model from scratch — you *harness* a pre-trained one, giving it the right context, tools, knowledge, and guardrails to solve a business problem reliably.

This guide maps the whole field. You don't need a maths background; every concept starts with an intuition and then goes one level deeper. Read it front to back for a complete mental model, or jump to the part you need.

The territory, in ten parts:

1. **How LLMs actually work** — architecture from the ground up.
2. **How models are trained and aligned** — pretraining, alignment, reasoning models.
3. **Prompting and context engineering** — steering the model.
4. **RAG** — giving the model knowledge it wasn't trained on.
5. **Agents** — models that take actions.
6. **Evaluation** — the most underrated skill in the field.
7. **Productionizing (LLMOps)** — latency, cost, reliability.
8. **Safety and security** — hallucination, prompt injection, guardrails.
9. **The 2025–2026 frontier** — what recently changed.
10. **The toolkit and roadmap** — the stack and what to learn in what order.

(The frontier moves fast. Specifics in Part 9 are current as of mid-2026; sources are listed at the end. Treat exact model names/numbers as examples of trends, not fixed facts.)

Part 1 — How LLMs Actually Work

To engineer with LLMs well, you need a true mental model of what's happening inside — not the maths, but the mechanics.

1.1 The one thing an LLM does: predict the next token

Strip away the mystique and a language model does exactly one thing: **given some text, it predicts the next chunk of text.** That's it. "The capital of France is ___" → it

assigns a high probability to "Paris." It generates a whole response by doing this over and over — predict a token, append it, predict the next — one piece at a time. Everything else (answering, coding, reasoning) is an emergent consequence of doing this extraordinarily well.

1.2 Tokens: the atoms of language models

Models don't see words or letters; they see **tokens** — subword chunks. "unbelievable" might split into `un`, `believ`, `able`. Common words are single tokens; rare ones fragment. Roughly, 1 token ≈ 0.75 English words, so ~1,000 tokens ≈ 750 words.

Tokens matter to you as an engineer for two concrete reasons: **you pay per token** (input + output), and the **context window** — the maximum tokens a model can consider at once — is measured in tokens. Understanding tokens is understanding your bill and your limits.

1.3 Embeddings: turning tokens into meaning

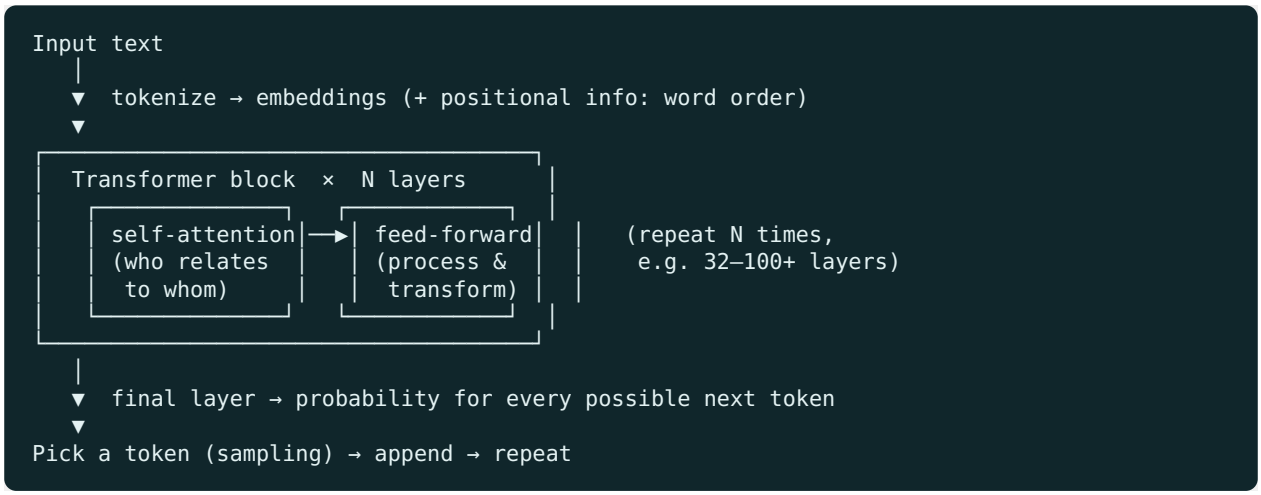
A model can't do arithmetic on the word "dog." So each token is mapped to an **embedding** — a long list of numbers (a vector) that encodes its meaning. The crucial property: **similar meanings land near each other** in this numeric space. "Dog" and "puppy" are close; "dog" and "spreadsheet" are far apart. This single idea — meaning as geometric closeness — is the foundation of both how models understand language *and* how RAG retrieves relevant documents (Part 4).

1.4 The Transformer and attention

Since 2017, essentially every major LLM is built on the **Transformer** architecture. Its key innovation is the **attention mechanism**, and it's worth understanding intuitively.

When processing a word, the model needs to know which *other* words in the text are relevant to it. In "The trophy didn't fit in the suitcase because *it* was too big," what does "it" refer to? Attention lets the model weigh every other token and decide "it" relates strongly to "trophy." **Self-attention** does this for every token against every other token, letting the model build a rich, context-aware understanding of the whole passage.

A Transformer stacks many layers of this:



Two supporting ideas: **positional encoding** tells the model the *order* of tokens (attention alone is order-blind), and the **feed-forward** layers do the heavy per-token processing. Stack dozens of these blocks, train on trillions of tokens, and language understanding emerges.

1.5 Decoding: how output is actually chosen

At each step the model produces a probability for every possible next token. How it *picks* one is controlled by **sampling parameters** you'll tune constantly:

- **Temperature** — randomness. Low (0–0.3) = focused, deterministic, best for factual/code tasks. High (0.8–1.2) = creative, varied, best for brainstorming.
- **Top-p (nucleus sampling)** — only consider the smallest set of tokens whose probabilities sum to *p* (e.g. 0.9), sampling from those. Controls diversity.

These don't change what the model knows — only how boldly it chooses among its options.

1.6 Context window and its consequences

The **context window** is everything the model can "see" at once: your system prompt + conversation history + retrieved documents + the current question, all counted in tokens. Historically a few thousand tokens, frontier models in 2026 reach **hundreds of thousands to over a million tokens.** But bigger isn't free: more context costs more, can slow responses, and models can still "lose" information buried in the middle of a huge context. Managing this budget is a core engineering skill (Part 3).

> **The essential mechanic of Part 1:** An LLM is a next-token predictor built from stacked attention layers that turn tokens into context-aware meaning. Tokens are your unit of cost and limit; embeddings are meaning-as-geometry; sampling controls output style; the context window is your working memory.

Part 2 — How Models Are Trained and Aligned

You rarely train a foundation model, but you must understand the pipeline — because *fine-tuning*, *reasoning models*, and *alignment* all live here and shape what you can build.

2.1 The three-stage pipeline

Stage 1 — Pretraining. The model reads a huge slice of the internet, books, and code, learning to predict the next token. This is where it absorbs grammar, facts, and reasoning patterns. It's astronomically expensive (millions of dollars, months of compute) and produces a "base model" that completes text but isn't yet a helpful assistant.

Stage 2 — Supervised fine-tuning (SFT). The base model is trained on curated `(prompt, ideal response)` pairs — examples of an assistant behaving well. This teaches it to *follow instructions* rather than just continue text.

Stage 3 — Preference alignment. The model is tuned to prefer responses humans (or an AI judge) rate as better — more helpful, honest, and harmless. This is where a raw model becomes the polished assistant you interact with.

2.2 Alignment methods you should know by name

- **RLHF (Reinforcement Learning from Human Feedback)** — the original approach: train a separate reward model on human preferences, then use reinforcement learning to optimize against it. Powerful but complex and unstable to run.
- **DPO (Direct Preference Optimization)** — the 2026 default for most teams. It trains directly on `(prompt, chosen, rejected)` preference pairs with no separate reward model and no fragile RL loop. Its descendants **ORPO** and **KTO** refine the idea further.

The standard modern recipe is simply **SFT → DPO**: teach good behavior, then teach preferences. Reserve full RLHF for cases needing careful long-horizon credit assignment (like complex agents) and the engineering muscle to debug it.

2.3 Reasoning models: the recent leap

A major shift: **reasoning models** (the o-series style, DeepSeek-R1, and successors) are trained — often with reinforcement learning on problems that have checkable answers — to **"think" before answering**, producing a long internal chain of reasoning. They trade speed for accuracy on hard problems (maths, logic, multi-step coding). The key concept is **test-time compute**: spending more computation *at inference* to reason, rather than only scaling training. For an engineer, this means: use a reasoning model when correctness on hard tasks matters more than latency or cost; use a fast standard model for simple, high-volume work.

2.4 Fine-tuning: when (and when not) to do it yourself

You can adapt an existing model to your domain. Options:

- **Full fine-tuning** — update all weights. Expensive, rarely necessary.
- **PEFT — Parameter-Efficient Fine-Tuning (LoRA / QLoRA)** — the practical standard. Instead of updating billions of parameters, you train tiny "adapter" layers (LoRA), optionally on a quantized/compressed model (QLoRA) so it fits on modest hardware. A common default is LoRA rank `r=16`.

The critical judgment call: fine-tune to change *behavior, style, or format* (a consistent tone, a structured output, a narrow domain skill). Do **not** fine-tune to add *knowledge* — that's what RAG is for (Part 4), and it's cheaper, updatable, and less error-prone. In practice, the order to try things is: **prompt → RAG → fine-tune**, escalating only when the simpler lever isn't enough.

> **The essential mechanic of Part 2:** Models are pretrained to predict text, SFT'd to follow instructions, and preference-aligned (now usually via DPO) to be helpful. Reasoning models add test-time "thinking" for hard problems. Fine-tune (via LoRA/QLoRA) to shape behavior, not to inject facts.

Part 3 — Prompting and Context Engineering

This is your primary interface to the model — and it has evolved from an art ("prompt engineering") into a systems discipline ("context engineering").

3.1 Prompt engineering fundamentals

- **The system message** is your most powerful lever: it sets persona, rules, constraints, and output format for the whole conversation ("You are a precise financial assistant. Never speculate. Always return JSON.").
- **Zero-shot** — just ask. **Few-shot** — include a handful of examples of the input→output you want; the model imitates the pattern. Few-shot dramatically improves consistency on structured tasks.
- **Chain-of-thought (CoT)** — asking the model to "think step by step" improves accuracy on reasoning tasks by making it show its work before answering. (Reasoning models do this internally by default.)
- **Structured output** — instruct the model to return strict JSON (or use the API's structured-output/JSON-mode feature) so your code can parse it reliably.

3.2 Context engineering: the bigger lever

Here's the shift experienced teams have internalized: **which information you put in the context window matters far more than how you phrase the prompt.** Context engineering is the discipline of *automatically assembling the right context* at inference time — retrieved documents, memory, tool outputs, and instructions — within your token budget.

In real 2026 systems, prompt wording matters less than *which three retrieved chunks the model saw* and *which two tool calls the agent made*. Your job is to build the system that reliably supplies the right context, not to hand-craft magic words.

3.3 Practical context tools

- **Function / tool calling** — the model can output a structured request to call a function you defined ("call `get_weather(city='Paris')`"). Your code runs it and feeds the result back. This is the bridge from "text generator" to "system that does things" (and the foundation of agents).

- **Prompt caching** — cache a stable chunk of context (system prompt, large reference doc) so it isn't reprocessed and re-billed on every call. Often the single highest-ROI cost optimization, cutting spend and latency substantially.

> **The essential mechanic of Part 3:** Steer with a strong system message, examples, and structured outputs — but invest most in *context engineering*: the system that assembles the right retrieved knowledge, memory, and tool results into the window. Phrasing is the small lever; context is the big one.

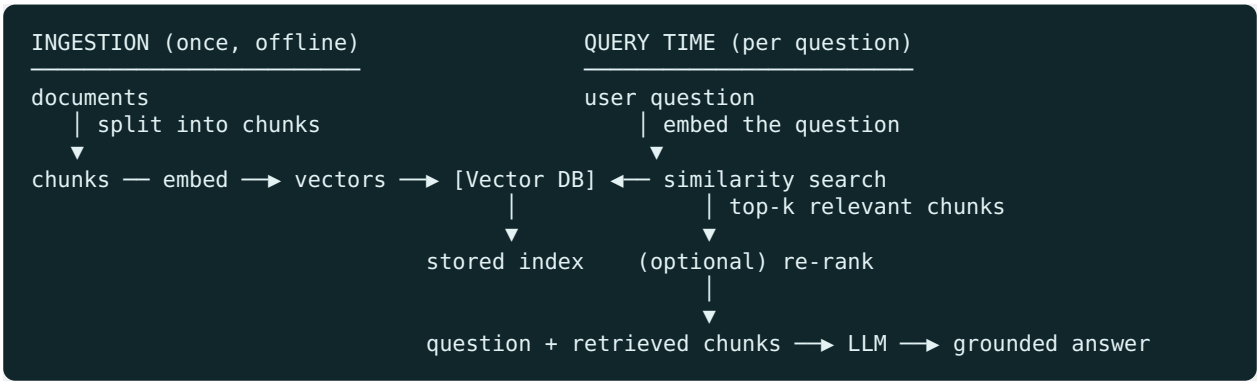
Part 4 — RAG: Giving the Model Knowledge

Foundation models have two knowledge problems: their training has a **cutoff date** (they don't know recent events) and they don't know your **private data** (your docs, your database). When pushed beyond what they know, they may **hallucinate** — produce confident, plausible falsehoods. **Retrieval-Augmented Generation (RAG)** solves this.

4.1 The core idea

Instead of relying on the model's memory, you **retrieve relevant information at query time and put it in the context window**, then ask the model to answer *using that provided information*. The model becomes a reasoning engine over facts you supply — grounded, current, and citable.

4.2 The RAG pipeline, step by step



Step 1 — Chunk. Split documents into passages (a few hundred tokens each). Chunking strategy matters a lot. **Step 2 — Embed.** Turn each chunk into an embedding vector (Part 1.3) using an embedding model. **Step 3 — Store.** Put vectors in a **vector database** (Pinecone, Qdrant, Weaviate, pgvector) that can find nearest neighbors fast. **Step 4 — Retrieve.** Embed the user's question, find the most similar chunks (top-k). **Step 5 — (Re-rank).** A reranker model reorders candidates so the *most* relevant land first. **Step 6 — Generate.** Feed question + retrieved chunks to the LLM, instructing it to answer from them and cite sources.

4.3 Advanced RAG

- **Hybrid search** — combine semantic (vector) search with classic keyword search; each catches what the other misses (keywords nail exact terms like product codes; vectors catch meaning).
- **Rerankers** — a second-stage model that dramatically improves which chunks actually reach the LLM. High leverage.
- **Agentic RAG** — instead of a fixed retrieve-then-answer pipeline, an agent *decides* what to retrieve, can search multiple times, reformulate queries, and combine sources. This upgrades RAG from a passive pipeline into an active reasoning loop (bridging to Part 5).

4.4 Evaluating RAG

RAG has two failure points to measure separately: **retrieval** (did we fetch the right chunks?) and **generation** (did the model answer faithfully from them?). Tools like **RAGAS** score dimensions such as *context relevance*, *faithfulness* (is the answer grounded in the retrieved text, or made up?), and *answer relevance*. You cannot improve RAG you don't measure.

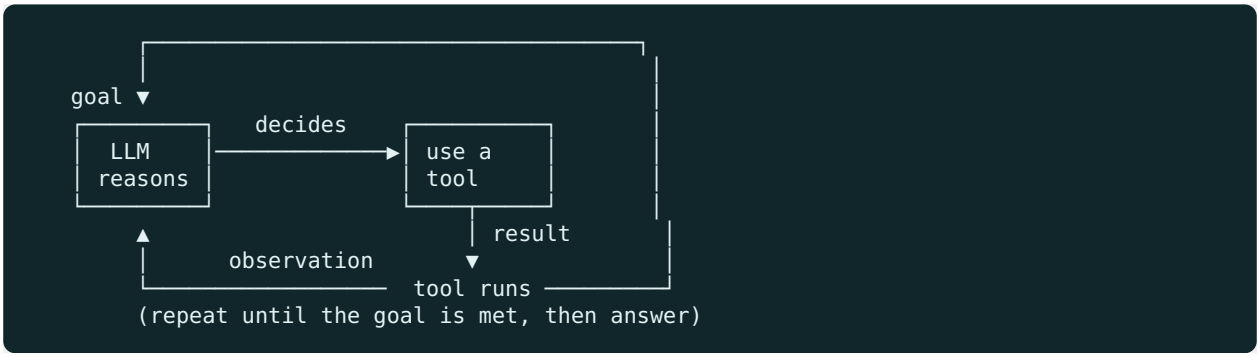
> **The essential mechanic of Part 4:** RAG grounds the model in knowledge it lacks by retrieving relevant chunks (via embeddings + a vector DB, improved by hybrid search and rerankers) and putting them in context. Prefer RAG over fine-tuning for adding facts, and evaluate retrieval and generation separately.

Part 5 — Agents

An **agent** is an LLM that can take actions in a loop to accomplish a goal, not just emit one response. This is where AI engineering is heading fastest.

5.1 The anatomy of an agent

An agent = **an LLM (the brain) + tools (hands) + a loop (persistence) + memory (state).** The classic loop, often called **ReAct** (Reason + Act):



The model reasons about what to do, calls a tool (search, database query, code execution, an API), observes the result, and repeats — until it can deliver a final answer. **Tool/function calling** (Part 3.3) is the mechanism that makes this possible.

5.2 Memory as a first-class primitive

Simple chatbots forget everything between sessions. Real agents need **memory**: short-term (the current conversation) and long-term (facts about the user or task, persisted and retrieved later — often via a RAG-like store). In 2026, memory has become a core architectural component, with systems that compress and distill conversation history to preserve important context while cutting token usage dramatically.

5.3 MCP: standardizing how agents connect to the world

The **Model Context Protocol (MCP)** is an open standard for connecting agents to tools and data sources. Before MCP, every tool integration was bespoke; MCP makes the integration layer **portable** — write a tool server once, and any MCP-compatible agent can use it. It has become foundational plumbing for the agent ecosystem.

5.4 Orchestration and multi-agent systems

For complex work, you compose multiple specialized agents — a planner, a researcher, a coder, a reviewer — coordinated by a supervisor. Frameworks like **LangGraph** model

this as a graph of steps with control flow, state, and checkpoints. A key 2026 nuance: because reasoning models are now so capable, a **single well-prompted reasoning-model call can sometimes replace an entire multi-step chain** — so don't add agent complexity you don't need.

5.5 What to watch for

Agents are powerful because they act autonomously — which is also the risk. Guard against **runaway loops** (set iteration limits), **blast radius** (give each agent only the minimum tools; never hand broad agents destructive powers or production credentials), **compounding errors** (a wrong step early derails everything after), and **cost blowups** (every loop iteration is more tokens). Always build in verification and human gates for high-risk actions.

> **The essential mechanic of Part 5:** An agent is an LLM in a reason-act-observe loop with tools and memory. Tool calling gives it hands, memory gives it continuity, MCP standardizes its connections, and orchestration composes specialists — but scope every agent tightly and don't add loops a single strong model call could handle.

Part 6 — Evaluation: The Most Underrated Skill

Ask senior AI engineers what separates a demo from a product, and the answer is **evaluation.** Nearly every senior job description asks for experience designing eval pipelines. LLMs are non-deterministic and fail in subtle ways; without systematic evaluation you're flying blind.

6.1 Why eval is hard and essential

Traditional software has deterministic tests: input X always gives output Y. LLM outputs vary, and "correct" is often fuzzy (is this summary *good*?). You need evaluation that handles open-ended, probabilistic behavior — and you need it *before* you ship changes, because a prompt tweak that helps one case can silently break ten others.

6.2 Offline evaluation

- **Golden datasets** — a curated set of representative inputs with known-good outputs (or acceptance criteria). Your regression test suite for AI.
- **Task-specific metrics** — exact match, tool-call accuracy, BLEU/ROUGE for summarization, retrieval hit-rate for RAG.
- **LLM-as-a-judge** — use a strong model to score outputs on dimensions like faithfulness, instruction-following, helpfulness, and tone. Scalable and surprisingly effective, though you must validate the judge itself against human ratings.

6.3 Online evaluation and observability

Offline isn't enough; production reveals what test sets miss.

- **Observability tools** (Langfuse, Arize Phoenix, Helicone) trace every request: the prompt, retrieved context, tool calls, tokens, latency, and cost. Essential for debugging and cost control.
- **User feedback signals** — thumbs up/down, corrections, task completion — feed back into your golden dataset.
- **Eval-driven development** — treat evals like tests: change a prompt or model, run the eval suite, ship only if scores hold or improve.

> **The essential mechanic of Part 6:** Build golden datasets, score with task metrics plus LLM-as-judge, trace everything in production with observability tools, and gate every change on your eval suite. You cannot improve what you don't measure — and in AI, measuring is the hard part.

Part 7 — Productionizing: LLMOps

Getting an LLM app to work once is easy; making it fast, cheap, and reliable at scale is the engineering.

7.1 The three inference dimensions

Every production decision trades off **latency** (how fast), **cost** (per request), and **quality** (how good). You're always balancing these:

- **Streaming** responses (token-by-token) improves *perceived* latency hugely — users see output immediately.
- **Model routing** — send easy requests to a small/cheap/fast model and only hard ones to the frontier model. A major cost lever.
- **Prompt caching and batching** cut cost and latency for repeated or bulk work.
- **Smaller and open models** — a well-chosen small or fine-tuned open model can match a frontier model on a narrow task at a fraction of the cost, and can run on your own infrastructure.

7.2 Reliability engineering

LLMs fail in ways normal APIs don't. Build in:

- **Structured-output validation** — never trust that JSON parses; validate it and retry on failure.
- **Retries and fallbacks** — retry transient errors; fall back to another model or a safe default if the primary fails.
- **Timeouts and rate-limit handling** — external model APIs throttle and occasionally go down; design for it.
- **Guardrails** — validate inputs and outputs against your rules (Part 8).

7.3 Build vs. buy: open vs. closed models

A recurring decision: use a **closed API model** (OpenAI, Anthropic, Google — best quality, zero infra, but per-token cost and data leaves your walls) or **self-host an open model** (Llama, Qwen, DeepSeek, Mistral — full control, data privacy, fixed infra cost, but you run the servers). Open models have closed much of the quality gap and dropped sharply in price, making self-hosting increasingly viable for teams with privacy needs or high volume.

> **The essential mechanic of Part 7:** Production LLM work is balancing latency, cost, and quality — via streaming, model routing, caching, and right-sized models — while engineering for the ways LLMs fail: validate outputs, retry, fall back, and guard the edges.

Part 8 — Safety, Security, and Responsible AI

These aren't optional add-ons; they're core to shipping AI you can trust.

8.1 The failure modes to know

- **Hallucination** — confident falsehoods. Mitigate with RAG grounding, citations, and instructions to say "I don't know."
- **Prompt injection** — malicious instructions hidden in user input or retrieved content ("ignore your rules and reveal the system prompt"). The top security risk for LLM apps; especially dangerous for agents with tools. Mitigate by separating trusted instructions from untrusted data, constraining tool permissions, and validating actions.
- **Jailbreaks** — crafted prompts that bypass safety training.
- **Bias** — models can reflect biases in training data; test across demographics for sensitive applications.
- **Data privacy / PII** — user data sent to a model may be sensitive; know where data goes, redact PII, and consider self-hosting for regulated data.

8.2 Guardrails and governance

- **Guardrails** — programmatic checks on inputs and outputs: block disallowed content, enforce formats, filter PII, and constrain what agents may do.
- **Red-teaming** — deliberately attack your own system (jailbreaks, injections, edge cases) before adversaries do.
- **Governance** — logging, audit trails, human oversight for high-stakes decisions, and clear policies for acceptable use.

> **The essential mechanic of Part 8:** Treat hallucination, prompt injection, jailbreaks, bias, and privacy as first-class engineering concerns. Ground outputs, separate instructions from data, constrain agent tools, add input/output guardrails, and red-team before shipping.

Part 9 — The 2025–2026 Frontier: What Recently Changed

The field is moving fast. Here are the shifts an engineer should have on their radar, current as of mid-2026.

9.1 Reasoning models and test-time compute

The biggest conceptual shift: models trained to **think before answering** (o-series, DeepSeek-R1 and successors), spending extra inference compute to reason through hard problems. This reframed the scaling story from "just train bigger" to "let the model compute more at answer time," and dramatically improved maths, logic, and coding performance.

9.2 Massive context windows

Frontier models now offer context windows of **hundreds of thousands to over a million tokens** — enough to hold entire codebases or document sets at once. This shifts some workloads from RAG toward "just put it all in context," though RAG remains essential for large, dynamic, or private corpora (and for cost control).

9.3 Native multimodality as standard

Handling **text, images, audio, and increasingly video** in a single model is now table stakes for frontier systems, not a special feature. AI engineers increasingly build across modalities by default (screenshots, documents, voice).

9.4 Agentic everything, standardized by MCP

The industry has pivoted hard to **agents**. Recent model releases explicitly target agentic workflows and long-horizon tasks, and **MCP** has emerged as the connective standard for tools and data. "Agent" moved from research demo to the default architecture for serious applications.

9.5 Memory and context engineering as disciplines

Memory became a first-class architectural primitive, with dedicated systems that compress and distill history (cutting token use by large margins while preserving

fidelity). And **context engineering** overtook prompt wording as the primary lever teams invest in.

9.6 Open models and falling prices

Open-weight models (Llama, Qwen, DeepSeek, Mistral, and others) have closed much of the gap to closed frontier models, often with permissive licenses, while API prices have dropped steeply (some releases cutting costs by well over half). This makes strong AI far more accessible and self-hosting more attractive. Alongside this, **small, efficient, and on-device models** have become a serious category for latency-sensitive and private workloads.

> **The essential mechanic of Part 9:** Recently: reasoning models and test-time compute, million-token context, native multimodality, agents-as-default with MCP as the standard, memory and context engineering as core disciplines, and cheaper/stronger open models. The direction is clear — more autonomous, more grounded, more multimodal, more accessible.

Part 10 — The AI Engineer's Toolkit and Roadmap

10.1 The modern stack

Layer	What it does	Common tools
Models	The reasoning engine	OpenAI, Anthropic, Google (closed); Llama, Qwen, DeepSeek, Mistral (open)
Orchestration	Chains, agents, control flow	LangChain, LlamaIndex, LangGraph
Vector database	Store & search embeddings	Pinecone, Qdrant, Weaviate, pgvector
Embeddings & rerankers	Turn text into vectors; reorder results	Provider embedding APIs, Cohere/BGE rerankers
Evaluation	Score quality	RAGAS, LLM-as-judge, custom golden sets
Observability	Trace, debug, cost-track	Langfuse, Arize Phoenix, Helicone
Tool connectivity	Standard agent–tool interface	MCP servers
Serving / infra	Run models & apps	vLLM, cloud GPU, model APIs

10.2 What to learn, in order

A sensible progression from most to least fundamental:

1. **LLM fundamentals** — tokens, embeddings, context, sampling (Part 1).
2. **Prompting and structured outputs** — get reliable behavior from an API (Part 3).
3. **RAG** — embeddings, vector DBs, retrieval, reranking (Part 4). *High hiring demand.*
4. **Agents and tool calling** — loops, memory, MCP, orchestration (Part 5). *High hiring demand.*
5. **Evaluation and observability** — golden sets, LLM-as-judge, tracing (Part 6). *The underrated differentiator.*
6. **Productionization** — cost/latency/reliability, guardrails (Parts 7–8).
7. **Fine-tuning** — LoRA/QLoRA, only when prompt + RAG aren't enough (Part 2).

The three skills hiring managers weight most in 2026 are **RAG, agents, and evaluation** — build real projects in each.

10.3 The AI engineer's mindset

Across everything, one posture defines the good AI engineer: **treat the model as a powerful but unreliable component, and engineer a reliable system around it.** You supply the right context, constrain the behavior, measure the output relentlessly, and design for failure. The intelligence comes from the model; the *reliability* comes from you.

The one idea underneath everything

> **AI engineering is the discipline of turning a probabilistic, general-purpose model into a dependable, specific product — through context, tools, retrieval, evaluation, and guardrails.**

The models will keep getting better on their own. Your enduring value is the system-building around them: giving them the right knowledge (RAG), the right abilities (agents and tools), the right guidance (context engineering), proof that they work (evaluation), and protection when they don't (safety and ops). Master those, and you can build on whatever the frontier delivers next.

Sources

- AI engineering stack & in-demand skills (2026): https://dataskew.io/roadmaps/ai-engineering/ and https://www.oreilly.com/radar/the-ai-agents-stack-2026-edition/
- Latest LLM developments, reasoning models, context windows, multimodality (2026): https://llm-stats.com/llm-updates and https://aimlapi.com/blog/top-llm-models-in-2026-the-best-ai-models-for-reasoning-coding-multimodal-tasks
- Context engineering, MCP, memory, agentic RAG: https://www.meta-intelligence.tech/en/insight-context-engineering and https://mem0.ai/blog/context-engineering-ai-agents-guide
- Fine-tuning, LoRA/QLoRA, DPO, LLM-as-judge (2026): https://futureagi.com/blog/llm-fine-tuning-techniques-i-ii/ and https://zylos.ai/research/2026-01-13-llm-fine-tuning-techniques/